PyTorch on GPUs

Single-GPU Training and Optimisation

- Week 13 – Advanced ML Computing
- Focus: Unlocking high performance AI workflows using GPU acceleration in PyTorch

Learning Objectives

By the end of this session, you will;

- Understand why GPU acceleration is critical in the AI/ML industry
- Configure PyTorch models and data to run on a GPU
- Optimise batching and data loading for GPU speed
- Recognise and troubleshoot common GPU issues
- Apply skills to real-world and assessment projects

Agenda

1. Review; Why run ML on GPUs
2. PyTorch GPU workflow; Device management and model migration
3. Optimising data loading and batching
4. Troubleshooting hardware and software issues
5. Hands-on activity; Move from CPU to GPU
6. Industry scenarios and assessment links

Industry Context

- Machine learning engineering and HPC rely on high-performance computing for rapid iteration
- GPU acceleration is expected for modern AI roles (model training, rapid prototyping, inference)
- Local software companies and government use GPU-based ML for computer vision, NLP, and predictive analytics
- Directly prepares you for working with cloud GPUs on Azure as required for this TAFE course and future work

Why GPU Acceleration Matters

- GPUs process thousands of operations in parallel
- Essential for deep learning (neural networks, massive data)
- Training time reduced from days to hours or minutes
- Industry expects GPU-fluent practitioners
- Rapid iteration = competitive advantage in deployed AI/ML solutions

PyTorch Overview; CPU vs GPU

- By default, PyTorch runs on CPU; works well for small tasks only
- GPUs require explicit configuration; major speedup for ML ops
- PyTorch uses CUDA (NVIDIA) for GPU acceleration
- Code must transfer tensors and models to GPU for computation

PyTorch Device Management

- Use `torch.cuda.is_available()` to check for GPU
- Define device variable
- Move tensors and models using `.to(device)`

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MyModel().to(device)
inputs = data.to(device)
```

- Always ensure both model and data are on the same device

Data and Batch Optimisation for GPU

- Batch size; Increase if using GPU for best throughput
- Use `DataLoader` with `num_workers` > 0 for parallel data loading
- Pin memory for fast transfer (`pin_memory=True`)
- Large batches may cause “CUDA out of memory”; tune to find optimal value

```
from torch.utils.data import DataLoader
loader = DataLoader(dataset, batch_size=64, shuffle=True, num_workers=4, pin_memory=True)
```

GPU Troubleshooting; Common Pitfalls

- CUDA not available: Check drivers and hardware compatibility
- “CUDA out of memory” errors: Reduce batch size or check for memory leaks
- Mismatched device errors: All tensors and models must be on same device
- Library/version mismatches: Align PyTorch and CUDA versions for your environment
- Use `torch.cuda.memory_summary()` for diagnostics

Activity; Run PyTorch on GPU

- Convert your Week 10 portfolio notebook to use GPU where available
- Steps;
  i. Check for GPU
  ii. Adjust device definitions
  iii. Move models and data onto device
  iv. Observe training time and compare to CPU run
- Record findings and discuss results; What changed. Any issues.

Example; Measuring Speedup

- Compare CPU vs GPU training loop

```
import time
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MyModel().to(device)
start = time.time()
for data, target in loader:
    data, target = data.to(device), target.to(device)
    output = model(data)
    loss = loss_fn(output, target)
    loss.backward()
    optimizer.step()
print(f"Elapsed time: {time.time() - start}")
```

- Measure and report elapsed time for both CPU and GPU
- Discuss why differences occur

Industry Example; Single-GPU Optimisation Matters

- Scenario; Start-up uses GPU VM on Azure to train fraud detection models nightly
- Optimised GPU code reduces model delivery from 8 hours to 1 hour
- Enables daily model refresh, faster business response, and cost savings in cloud billing
- Documentation and monitoring (Week 11) ensure reliable GPU operations

Assessment Preparation

- This content underpins the “Run a pytorch script on a single GPU” project
- Ensure you can;
  - Detect GPUs and select devices
  - Move code from CPU to GPU
  - Optimise data pipelines for performance
  - Debug and troubleshoot CUDA errors
- Practice and share results in your portfolio notebook

Key Takeaways

- GPU acceleration is a must-have skill for AI/ML in industry
- PyTorch needs explicit code changes for GPU use
- Data loading and batching majorly impact performance
- Troubleshooting is a daily part of ML engineering
- Skills from today feed into multi-GPU and cloud workflows (Weeks 15–18)

Next Steps

- Complete today’s activity; update and run your portfolio code on GPU
- Prepare questions/issues for troubleshooting workshop next session
- Read ahead: Distributed GPU training and torchrun (Week 15)

References and Acknowledgements

- PyTorch Documentation: “Deep Learning with PyTorch: A 60 Minute Blitz” (link)
- PyTorch Data Parallelism Tutorial (link)
- NMTAFE AI/ML Course Industry Context (2024)